

STUDENT CASE STUDY EXECUTION REPORT

DEPARTMENT	FED
SUBJECT	DSA-2
ACADEMIC YEAR	I-III Semester (2025-26)
FACULTY NAME	Mohammed Razia Alangir Banu
STUDENT NAME	A. Sai Kiran
ROLL NUMBER	2520030125
BRANCH	Bachupally
COURSE OUTCOME	

➤ **Case Study Topic:** Foodchain Backend Restaurant & delivery Index Management using Binary search tree (BST) & self-balancing AVL Trees.

➤ **Work Done Summary:**

In the **FoodChain – Smart Food Delivery & Restaurant Logistics System**, we implemented **Binary Search Trees (BST)** and **self-balancing AVL Trees** to manage hierarchical datasets such as **restaurant Records, menu item IDs, order files, and delivery rider data**.

BST was used to **construct, insert, delete, and search** indexed data in sorted form. However, unbalanced insertions reduced efficiency.

To solve this, an **AVL Tree** was implemented to maintain balance using **rotation operations (LL, RR, LR, RL)** and ensure **$O(\log n)$** performance for all operations.

This makes the system efficient for **file indexing and dictionary-style search operations** in real-time food delivery systems.

➤ **Implementation Details**

1. Inserted restaurant records, delivery rider IDs, and menu item indexes into a **Binary Search Tree (BST)** Based on unique identifiers to maintain structured ordering.
2. Constructed hierarchical indexing for **FoodChain backend data to support efficient restaurant lookup, Order tracking, and menu retrieval**.
3. Performed search operations in BST to retrieve specific records using key-based traversal, ensuring fast Access to required data.

4. Detected performance degradation in BST due to skewed insertions and converted the structure into a **self-balancing AVL Tree for optimization**.
5. Implemented AVL Tree balancing using balance factor checks **and rotations (LL, RR, LR, RL)** to maintain Optimal tree height during insertions and deletions.
6. Achieved improved system performance with AVL Tree ensuring $O(\log n)$ time complexity for insertion, Deletion, and search operations, making it suitable for large-scale FoodChain backend indexing.

➤ **RESULT: Screenshot/Output**

```
PS C:\Users\Sai\OneDrive\Desktop\DSA case study> javac FoodChainAVL.java ; java FoodChainAVL
AVL INSERTION (Arrival Order)
Insertion order:
45, 25, 60, 15, 35, 55, 75, 10, 20
Rotations that occurred:
None (Tree remained balanced)
FINAL AVL TREE (Descending by timestamp)

          45(bf=+1)
         /      \
      60(bf=0)   25(bf=+1)
     /  \       /  \
  75(bf=0) 55(bf=0) 35(bf=0) 20(bf=0)
 /  \
15(bf=0)
 \
  10(bf=0)

TOP 5 DESCENDING (k = 5)
Top 5 timestamps (most recent first):
[75, 60, 55, 45, 35]
Time Complexity (worst case):  $O(\min(n, k) + \log n)$ 
```

The AVL Tree Rotation(LL, RR, RL, LR), insertion/deletion, searching, balance factor&.....etc.

Top 9 recent timestamps returned were:

45, 25, 60, 15, 35, 55, 75, 10, 20

Time Complexity:

- Search, Insert, Delete- **$O(\log n)$**
- Height Calculation, Balance Factor Calculation, AVL Rotation (LL, RR, LR, RL)- $O(1)$

The AVL Tree efficiently managed restaurant records in the FoodChain backend system. By maintaining a balanced structure, it guaranteed **$O(\log n)$** search, insertion, and deletion operations, resulting in faster restaurant indexing, quick data retrieval, and improved overall system performance compared to a normal Binary Search Tree.

GitHub link: <https://github.com/slashnight/DSA-2-case-study-assignment>

Student Signature	Faculty Signature

